⬚ MASTER PROMPT — 3D Interactive Real Estate Plot Booking Website

(Like Spacer.land / Preethi Estates)

OVERVIEW

Build a **single-file, fully responsive 3D interactive real estate plot booking website** called **"Preethi Estates – Premium Plotting Collection"** using pure HTML, CSS, and JavaScript with **Three.js** for 3D rendering. The website must work flawlessly on any device — desktop, tablet, and mid-range Android/iOS phones — with **zero overlapping elements**, **auto-adjusting button sizes**, and **smooth 3D/2D interactions**.

Reference style: spacer.land — a premium dark-themed 3D plot visualization platform.

TECH STACK

- **Single HTML file** (`preethi-estates.html`) — all CSS and JS embedded
- **Three.js r158** via CDN for 3D rendering
- **OrbitControls** from Three.js addons for camera pan/zoom/rotate
- **Google Fonts:** `Inter` (body), `Syne` or `Space Grotesk` (headings/labels)
- **No frameworks, no build tools, no external dependencies except Three.js CDN**
- All assets inline (SVG icons, CSS gradients, no external images needed)

DESIGN SYSTEM — DARK LUXURY THEME

```
:root {
  /* Dark surfaces */
  --color-bg: #0f0f0f;
  --color-surface: #161616;
  --color-surface-2: #1e1e1e;
  --color-surface-3: #252525;
  --color-border: rgba(255,255,255,0.08);
  --color-divider: rgba(255,255,255,0.05);

  /* Text */
  --color-text: #f0efed;
  --color-text-muted: #8a8986;
```

```
  --color-text-faint: #4a4947;
  --color-text-inverse: #0f0f0f;

  /* Plot Status Colors */
  --color-available: #22c55e;       /* Green */
  --color-sold: #ef4444;            /* Red */
  --color-corner: #eab308;          /* Yellow/Gold — corner/road-facing */
  --color-selected: #38bdf8;        /* Sky blue — active selection */
  --color-road: #2a2a2a;            /* Road surface */
  --color-road-line: #4a4a4a;       /* Road marking lines */

  /* UI Accent */
  --color-accent: #3b82f6;          /* Blue — BOOK NOW button */
  --color-accent-hover: #2563eb;
  --color-cta-text: #ffffff;

  /* Status badge colors */
  --badge-available-bg: rgba(34, 197, 94, 0.15);
  --badge-available-text: #4ade80;
  --badge-sold-bg: rgba(239, 68, 68, 0.15);
  --badge-sold-text: #f87171;

  /* Fluid type scale */
  --text-xs: clamp(0.75rem, 0.7rem + 0.25vw, 0.875rem);
  --text-sm: clamp(0.8rem, 0.75rem + 0.3vw, 0.9375rem);
  --text-base: clamp(0.875rem, 0.8rem + 0.35vw, 1rem);
  --text-lg: clamp(1rem, 0.9rem + 0.5vw, 1.25rem);
  --text-xl: clamp(1.2rem, 1rem + 1vw, 1.75rem);

  /* Spacing */
  --space-1: 0.25rem;  --space-2: 0.5rem;  --space-3: 0.75rem;
  --space-4: 1rem;     --space-5: 1.25rem; --space-6: 1.5rem;
  --space-8: 2rem;

  /* Radius */
  --radius-sm: 0.375rem;
  --radius-md: 0.5rem;
  --radius-lg: 0.75rem;
  --radius-xl: 1rem;
  --radius-full: 9999px;

  /* Shadows */
  --shadow-panel: 0 8px 32px rgba(0,0,0,0.6);
  --shadow-button: 0 2px 8px rgba(0,0,0,0.4);

  /* Transitions */
  --transition: 200ms cubic-bezier(0.16, 1, 0.3, 1);
}
```

LAYOUT ARCHITECTURE

Full-Screen Canvas Layout (All layers use `position: fixed`)

```
┌──────────────────────────────────────────────────────┐
│ [TOP BAR]                            [SHARE BTN]  │ ← z-index: 100
├──────────────────────────────────────────────────────┤
│                                                      │
│          THREE.JS CANVAS (full-screen)               │ ← z-index: 1
│                                      ┌──────────────┐ │
│                                      │ PLOT INFO    │ │ ← z-index: 200
│                                      │   PANEL      │ │
│                                      └──────────────┘ │
│                           [2D][3D][SD]               │ ← z-index: 100
│                                                      │
├──────────────────────────────────────────────────────┤
│ [STATUS] [SEARCH BAR ──────────] [──────────────]    │ ← z-index: 100
│          [GALLERY] [INFO] [LOCATE]                   │ ← z-index: 100
└──────────────────────────────────────────────────────┘
```

CRITICAL RESPONSIVE RULES (no overlapping):

- All UI layers use `position: fixed` with specific `z-index` values
- Each UI region has its own `safe-area-inset` awareness
- On mobile: Plot Info Panel slides up from bottom (not side panel)
- Buttons use `clamp()` for width/height/font-size at all breakpoints
- Bottom controls stack intelligently — no overlap with plot panel
- Use CSS Grid for the bottom bar layout on all screen sizes

HTML STRUCTURE

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1
  <meta name="theme-color" content="#0f0f0f" />
  <title>Preethi Estates — Premium Plotting Collection</title>
  <!-- Fonts -->
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300..700&family=Space+
  <!-- Three.js CDN -->
  <script src="https://cdnjs.cloudflare.com/ajax/libs/three.js/r128/three.min.js"></scri
  <!-- OrbitControls — must be loaded AFTER three.js -->
  <style> /* ALL CSS HERE */ </style>
</head>
<body>
  <!-- 1. TOP BAR -->
```

```
<header class="top-bar" role="banner">
  <div class="brand">
    <svg class="brand-logo" ...></svg>  <!-- Custom SVG logo -->
    <div class="brand-text">
      <span class="brand-name">PREETHI ESTATES</span>
      <span class="brand-sub">PREMIUM PLOTTING COLLECTION</span>
    </div>
  </div>
  <button class="btn-share" aria-label="Share this project">
    <svg><!-- share icon --></svg> SHARE
  </button>
</header>

<!-- 2. THREE.JS CANVAS CONTAINER -->
<div id="canvas-container" aria-label="3D plot map" role="img">
  <canvas id="plot-canvas"></canvas>
</div>

<!-- 3. PLOT INFO PANEL (right side desktop, bottom sheet mobile) -->
<aside id="plot-panel" class="plot-panel" aria-live="polite" hidden>
  <button class="panel-close" aria-label="Close plot info">×</button>
  <div class="panel-plot-number">Plot #<span id="panel-number">--</span></div>
  <div id="panel-status-badge" class="status-badge status-available">
    <span class="status-dot"></span>
    <span id="panel-status-text">AVAILABLE</span>
  </div>
  <div class="panel-details">
    <div class="panel-detail-item">
      <span class="detail-label">AREA</span>
      <span class="detail-value"><strong id="panel-area">--</strong> <small>sq ft</sma
    </div>
    <div class="panel-detail-item">
      <span class="detail-label">PRICE</span>
      <span class="detail-value price"><strong id="panel-price">--</strong></span>
    </div>
  </div>
  <div class="panel-actions">
    <button class="btn-book" id="btn-book-now">BOOK NOW</button>
    <button class="btn-contact" id="btn-contact">CONTACT</button>
  </div>
  <div class="panel-meta">
    <div class="meta-item"><span>Facing</span><span id="panel-facing">--</span></div>
    <div class="meta-item"><span>Dimensions</span><span id="panel-dims">--</span></di
    <div class="meta-item"><span>Road Width</span><span id="panel-road">--</span></di
  </div>
</aside>

<!-- 4. VIEW TOGGLE (2D / 3D / SD) -->
<div class="view-toggle" role="group" aria-label="View mode">
  <button class="view-btn" data-view="2d" aria-pressed="false">2D</button>
  <button class="view-btn active" data-view="3d" aria-pressed="true">3D</button>
  <button class="view-btn" data-view="sd" aria-pressed="false">SD</button>
</div>

<!-- 5. RESET CAMERA BUTTON -->
<button class="btn-reset-cam" aria-label="Reset camera view" title="Reset view">
```

```
    <svg><!-- reset/refresh icon --></svg>
  </button>

  <!-- 6. BOTTOM CONTROLS -->
  <div class="bottom-controls">
    <!-- Row 1: Status legend + Search -->
    <div class="bottom-row-1">
      <div class="status-legend">
        <button class="legend-toggle" aria-label="Toggle status legend">
          STATUS
          <div class="legend-indicator" id="legend-dot"></div>
        </button>
        <div class="legend-popup" id="legend-popup" hidden>
          <div class="legend-item"><span class="legend-color" style="background:var(--co
          <div class="legend-item"><span class="legend-color" style="background:var(--co
          <div class="legend-item"><span class="legend-color" style="background:var(--co
        </div>
      </div>
      <div class="search-wrapper">
        <svg class="search-icon"><!-- search icon --></svg>
        <input
          type="search"
          id="plot-search"
          class="search-input"
          placeholder="Search Plot Number..."
          aria-label="Search plot by number"
          inputmode="numeric"
          autocomplete="off"
        />
        <div id="search-results" class="search-results" hidden></div>
      </div>
    </div>
    <!-- Row 2: Bottom action tabs -->
    <div class="bottom-tabs" role="tablist">
      <button class="tab-btn" role="tab" data-tab="gallery" aria-selected="false">
        <svg><!-- gallery/photo icon --></svg> GALLERY
      </button>
      <button class="tab-btn" role="tab" data-tab="info" aria-selected="false">
        <svg><!-- info circle icon --></svg> INFO
      </button>
      <button class="tab-btn" role="tab" data-tab="locate" aria-selected="false">
        <svg><!-- location pin icon --></svg> LOCATE
      </button>
    </div>
  </div>

  <!-- 7. MODAL OVERLAYS (Gallery, Info, Locate) -->
  <div id="modal-overlay" class="modal-overlay" hidden role="dialog" aria-modal="true">
    <div class="modal-card">
      <button class="modal-close" aria-label="Close modal">×</button>
      <div id="modal-content"></div>
    </div>
  </div>

  <!-- 8. TOAST NOTIFICATION -->
  <div id="toast" class="toast" role="alert" aria-live="polite"></div>
```

```
  <script>
    // ═══ ALL JAVASCRIPT HERE ═══
  </script>
</body>
</html>
```

PLOT DATA STRUCTURE

```
const PLOT_DATA = [
  // Format: { id, number, status, area, price, facing, dims, roadWidth, row, col, isCo
  // status: 'available' | 'sold' | 'corner'
  // roadFacing: true/false — plots adjacent to roads get special road-side styling
  { id: 1,  number: 30,  status: 'available', area: 1200, price: '₹260K', facing: 'East',
  { id: 2,  number: 31,  status: 'available', area: 1200, price: '₹265K', facing: 'East',
  { id: 3,  number: 38,  status: 'available', area: 1050, price: '₹200K', facing: 'West',
  { id: 4,  number: 40,  status: 'sold',      area: 1050, price: '₹210K', facing: 'North
  { id: 5,  number: 42,  status: 'available', area: 1050, price: '₹215K', facing: 'North
  { id: 6,  number: 43,  status: 'available', area: 1100, price: '₹220K', facing: 'North
  { id: 7,  number: 44,  status: 'available', area: 1100, price: '₹225K', facing: 'North
  { id: 8,  number: 49,  status: 'available', area: 1000, price: '₹195K', facing: 'West',
  { id: 9,  number: 52,  status: 'corner',    area: 1300, price: '₹280K', facing: 'East',
  { id: 10, number: 53,  status: 'sold',      area: 1050, price: '₹210K', facing: 'North
  { id: 11, number: 54,  status: 'sold',      area: 1100, price: '₹215K', facing: 'North
  { id: 12, number: 56,  status: 'available', area: 1050, price: '₹215K', facing: 'North
  { id: 13, number: 57,  status: 'available', area: 1050, price: '₹215K', facing: 'North
  { id: 14, number: 58,  status: 'available', area: 1100, price: '₹220K', facing: 'East',
  { id: 15, number: 59,  status: 'available', area: 1050, price: '₹215K', facing: 'East',
  // Add more plots as needed...
];

// Road definitions — placed BETWEEN plot rows/columns
const ROADS = [
  { label: '12 Meter Road', direction: 'horizontal', position: 0,    width: 12 },
  { label: '9 Meter Road',  direction: 'vertical',   position: 3,    width: 9  },
];
```

THREE.JS 3D SCENE — COMPLETE IMPLEMENTATION

Scene Setup

```
// Scene constants
const PLOT_WIDTH  = 3.5;   // Three.js units per plot
const PLOT_HEIGHT = 0.25;  // Plot box height (3D elevation)
const PLOT_DEPTH  = 3.0;
const ROAD_WIDTH_12M = 2.5;
const ROAD_WIDTH_9M  = 1.8;
const PLOT_GAP    = 0.15;  // Gap between plots
const PLOT_HOVER_HEIGHT = 0.45; // Hover elevation
```

```
// Colors (Three.js hex)
const COLORS = {
  available:  0x22c55e,
  sold:       0xef4444,
  corner:     0xeab308,
  selected:   0x38bdf8,
  road:       0x1a1a1a,
  ground:     0x111111,
  tree_trunk: 0x5c3d11,
  tree_leaf:  0x16a34a,
  tree_leaf2: 0xec4899,  // pink tree variant
  tree_leaf3: 0x9333ea,  // purple tree variant
};

// ═══ SCENE INITIALIZATION ═══
function initScene() {
  const scene    = new THREE.Scene();
  scene.background = new THREE.Color(0x0f0f0f);
  scene.fog = new THREE.FogExp2(0x0f0f0f, 0.035);

  const camera = new THREE.PerspectiveCamera(
    45,
    canvas.clientWidth / canvas.clientHeight,
    0.1,
    200
  );
  camera.position.set(0, 18, 20);
  camera.lookAt(0, 0, 0);

  const renderer = new THREE.WebGLRenderer({
    canvas: document.getElementById('plot-canvas'),
    antialias: window.devicePixelRatio < 2,  // No MSAA on high-DPR mobile (performance
    powerPreference: 'default'
  });
  renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
  renderer.setSize(window.innerWidth, window.innerHeight);
  renderer.shadowMap.enabled = true;
  renderer.shadowMap.type = THREE.PCFSoftShadowMap;
  renderer.toneMapping = THREE.ACESFilmicToneMapping;
  renderer.toneMappingExposure = 1.2;

  return { scene, camera, renderer };
}
```

Lighting

```
function setupLighting(scene) {
  // Ambient
  const ambient = new THREE.AmbientLight(0xffffff, 0.4);
  scene.add(ambient);

  // Main directional (sun)
  const sun = new THREE.DirectionalLight(0xffffff, 1.2);
  sun.position.set(15, 25, 10);
  sun.castShadow = true;
```

```
  sun.shadow.mapSize.width  = 1024;  // NOT 2048+ — performance on mobile
  sun.shadow.mapSize.height = 1024;
  sun.shadow.camera.near = 1;
  sun.shadow.camera.far  = 80;
  sun.shadow.camera.left = sun.shadow.camera.bottom = -25;
  sun.shadow.camera.right = sun.shadow.camera.top   = 25;
  sun.shadow.bias = -0.0005;
  scene.add(sun);

  // Fill light
  const fill = new THREE.DirectionalLight(0x8bb5ff, 0.3);
  fill.position.set(-10, 10, -10);
  scene.add(fill);

  // Hemisphere
  const hemi = new THREE.HemisphereLight(0x1a1a2e, 0x0f0f0f, 0.5);
  scene.add(hemi);
}
```

Ground Plane

```
function createGround(scene) {
  const geo = new THREE.PlaneGeometry(80, 80);
  const mat = new THREE.MeshStandardMaterial({
    color: 0x111111,
    roughness: 0.95,
    metalness: 0.0,
  });
  const ground = new THREE.Mesh(geo, mat);
  ground.rotation.x = -Math.PI / 2;
  ground.receiveShadow = true;
  scene.add(ground);
}
```

Road System — PLOTS ON THE ROAD

```
function createRoads(scene) {
  const roadMat = new THREE.MeshStandardMaterial({
    color: 0x1e1e1e,
    roughness: 0.9,
    metalness: 0.0
  });
  const lineMat = new THREE.MeshStandardMaterial({ color: 0x3a3a3a });

  // 12 Meter Road — horizontal (runs across top of plot grid)
  const road12 = new THREE.Mesh(
    new THREE.BoxGeometry(40, 0.05, ROAD_WIDTH_12M),
    roadMat
  );
  road12.position.set(0, 0.01, -6);  // Position ABOVE the plot grid
  road12.receiveShadow = true;
  scene.add(road12);
```

```
  // Road dashed center line (12M road)
  for (let x = -18; x < 18; x += 1.5) {
    const dash = new THREE.Mesh(
      new THREE.BoxGeometry(0.8, 0.06, 0.07),
      lineMat
    );
    dash.position.set(x, 0.02, -6);
    scene.add(dash);
  }

  // 9 Meter Road — vertical (runs along the right side)
  const road9 = new THREE.Mesh(
    new THREE.BoxGeometry(ROAD_WIDTH_9M, 0.05, 30),
    roadMat
  );
  road9.position.set(8, 0.01, 2);  // Position to the right of plots
  road9.receiveShadow = true;
  scene.add(road9);

  // Road labels using CSS2DRenderer (or canvas texture sprites)
  addRoadLabel(scene, '12 Meter Road', { x: 5, y: 0.5, z: -6  }, 'horizontal');
  addRoadLabel(scene, '9 Meter Road',  { x: 8, y: 0.5, z: 2   }, 'vertical');
}

// Road label as a canvas texture sprite (works without CSS2DRenderer)
function addRoadLabel(scene, text, position, direction) {
  const canvas2d = document.createElement('canvas');
  canvas2d.width  = 512;
  canvas2d.height = 64;
  const ctx = canvas2d.getContext('2d');
  ctx.fillStyle = 'transparent';
  ctx.clearRect(0, 0, 512, 64);
  ctx.font = 'bold 28px Space Grotesk, Inter, sans-serif';
  ctx.fillStyle = 'rgba(255,255,255,0.4)';
  ctx.letterSpacing = '4px';
  ctx.textAlign = 'center';
  ctx.textBaseline = 'middle';
  ctx.fillText(text, 256, 32);

  const texture = new THREE.CanvasTexture(canvas2d);
  const mat = new THREE.SpriteMaterial({ map: texture, transparent: true, depthWrite: fa
  const sprite = new THREE.Sprite(mat);
  sprite.position.set(position.x, position.y, position.z);
  sprite.scale.set(6, 0.75, 1);
  if (direction === 'vertical') sprite.material.rotation = -Math.PI / 2;
  scene.add(sprite);
}
```

Plot Grid — PLOTS ADJACENT TO ROADS

```
// CRITICAL: Plot rows nearest to roads must be positioned so they
// visually touch the road edge. Use roadFacing flag to adjust z-position.

function buildPlotGrid(scene, plotData) {
  const plotMeshes = [];
```

```
  // Calculate layout origin so road-facing plots sit AT the road edge
  const ORIGIN_X = -12;
  const ORIGIN_Z = -4.5;  // This puts row 0 plots flush with the 12M road edge

  plotData.forEach(plot => {
    const x = ORIGIN_X + plot.col * (PLOT_WIDTH + PLOT_GAP);
    const z = ORIGIN_Z + plot.row * (PLOT_DEPTH + PLOT_GAP);

    const color = COLORS[plot.status] || COLORS.available;

    // Plot box geometry
    const geo = new THREE.BoxGeometry(PLOT_WIDTH, PLOT_HEIGHT, PLOT_DEPTH);
    const mat = new THREE.MeshStandardMaterial({
      color:     color,
      roughness: 0.7,
      metalness: 0.1,
      emissive:  new THREE.Color(color),
      emissiveIntensity: 0.08,
    });

    const mesh = new THREE.Mesh(geo, mat);
    mesh.position.set(x, PLOT_HEIGHT / 2, z);
    mesh.castShadow    = true;
    mesh.receiveShadow = true;
    mesh.userData = { plotId: plot.id, plotData: plot, originalY: PLOT_HEIGHT / 2, origi

    // Plot number label (sprite)
    addPlotLabel(scene, plot.number.toString(), { x, y: PLOT_HEIGHT + 0.05, z });

    scene.add(mesh);
    plotMeshes.push(mesh);
  });

  return plotMeshes;
}

function addPlotLabel(scene, text, position) {
  const c = document.createElement('canvas');
  c.width  = 128;
  c.height = 128;
  const ctx = c.getContext('2d');
  ctx.clearRect(0, 0, 128, 128);
  ctx.font = 'bold 52px Space Grotesk, Inter, sans-serif';
  ctx.fillStyle = 'rgba(0,0,0,0.8)';
  ctx.textAlign = 'center';
  ctx.textBaseline = 'middle';
  ctx.fillText(text, 64, 64);

  const texture = new THREE.CanvasTexture(c);
  const mat  = new THREE.SpriteMaterial({ map: texture, transparent: true, depthWrite: f
  const sprite = new THREE.Sprite(mat);
  sprite.position.set(position.x, position.y, position.z);
  sprite.scale.set(1.8, 1.8, 1);
  scene.add(sprite);
}
```

Trees (Decorative)

```
function createTree(scene, x, z, variant = 0) {
  const variants = [
    { leaf: COLORS.tree_leaf,  size: 0.6 },
    { leaf: COLORS.tree_leaf2, size: 0.5 },
    { leaf: COLORS.tree_leaf3, size: 0.55 },
  ];
  const v = variants[variant % variants.length];

  // Trunk
  const trunk = new THREE.Mesh(
    new THREE.CylinderGeometry(0.07, 0.1, 0.6, 6),
    new THREE.MeshStandardMaterial({ color: COLORS.tree_trunk })
  );
  trunk.position.set(x, 0.3, z);
  trunk.castShadow = true;
  scene.add(trunk);

  // Canopy — teardrop/cone shape using a ConeGeometry
  const canopy = new THREE.Mesh(
    new THREE.ConeGeometry(v.size, v.size * 2.2, 7),
    new THREE.MeshStandardMaterial({
      color: v.leaf,
      roughness: 0.8,
      emissive: new THREE.Color(v.leaf),
      emissiveIntensity: 0.15,
    })
  );
  canopy.position.set(x, 0.6 + v.size * 1.1, z);
  canopy.castShadow = true;
  scene.add(canopy);
}

// Place trees along roads and between plots
function placeTrees(scene) {
  const treePositions = [
    [-8, -7.5], [-4, -7.5], [0, -7.5], [4, -7.5], [8, -7.5],  // Along 12M road top
    [9.5, -2], [9.5, 2], [9.5, 6], [9.5, 10],                  // Along 9M road right
    [-14, 2], [-14, 6],                                         // Left edge
  ];
  treePositions.forEach(([x, z], i) => createTree(scene, x, z, i % 3));
}
```

Raycasting — Click/Tap to Select Plots

```
function setupInteraction(camera, renderer, plotMeshes, scene) {
  const raycaster = new THREE.Raycaster();
  const pointer   = new THREE.Vector2();
  let hoveredMesh = null;
  let selectedMesh = null;

  // Unified pointer handler (mouse + touch)
  function getPointerNDC(event) {
```

```
  const rect = renderer.domElement.getBoundingClientRect();
  const clientX = event.touches ? event.touches[^1_0].clientX : event.clientX;
  const clientY = event.touches ? event.touches[^1_0].clientY : event.clientY;
  return {
    x:  ((clientX - rect.left) / rect.width)  * 2 - 1,
    y: -((clientY - rect.top)  / rect.height) * 2 + 1,
  };
}

// Hover (desktop only)
renderer.domElement.addEventListener('mousemove', (e) => {
  const { x, y } = getPointerNDC(e);
  pointer.set(x, y);
  raycaster.setFromCamera(pointer, camera);
  const hits = raycaster.intersectObjects(plotMeshes);

  if (hoveredMesh && hoveredMesh !== selectedMesh) {
    animatePlotElevation(hoveredMesh, hoveredMesh.userData.originalY, 200);
    hoveredMesh.material.emissiveIntensity = 0.08;
  }

  if (hits.length > 0) {
    hoveredMesh = hits[^1_0].object;
    if (hoveredMesh !== selectedMesh) {
      animatePlotElevation(hoveredMesh, hoveredMesh.userData.originalY + 0.15, 200);
      hoveredMesh.material.emissiveIntensity = 0.25;
    }
    renderer.domElement.style.cursor = 'pointer';
  } else {
    hoveredMesh = null;
    renderer.domElement.style.cursor = 'grab';
  }
});

// Click / Touch to select
let touchStartPos = null;
renderer.domElement.addEventListener('touchstart', (e) => {
  touchStartPos = { x: e.touches[^1_0].clientX, y: e.touches[^1_0].clientY };
}, { passive: true });

function handleSelect(event) {
  // Distinguish drag from tap on touch
  if (event.type === 'touchend' && touchStartPos) {
    const dx = Math.abs(event.changedTouches[^1_0].clientX - touchStartPos.x);
    const dy = Math.abs(event.changedTouches[^1_0].clientY - touchStartPos.y);
    if (dx > 10 || dy > 10) return;  // Was a drag, not a tap
  }

  const clientX = event.changedTouches ? event.changedTouches[^1_0].clientX : event.cli
  const clientY = event.changedTouches ? event.changedTouches[^1_0].clientY : event.cli
  const rect = renderer.domElement.getBoundingClientRect();
  pointer.set(
    ((clientX - rect.left) / rect.width)  * 2 - 1,
    -((clientY - rect.top)  / rect.height) * 2 + 1
  );
```

```
    raycaster.setFromCamera(pointer, camera);
    const hits = raycaster.intersectObjects(plotMeshes);

    // Deselect previous
    if (selectedMesh) {
      selectedMesh.material.color.set(selectedMesh.userData.originalColor);
      selectedMesh.material.emissive.set(selectedMesh.userData.originalColor);
      selectedMesh.material.emissiveIntensity = 0.08;
      animatePlotElevation(selectedMesh, selectedMesh.userData.originalY, 300);
      selectedMesh = null;
    }

    if (hits.length > 0) {
      selectedMesh = hits[^1_0].object;
      selectedMesh.material.color.set(COLORS.selected);
      selectedMesh.material.emissive.set(COLORS.selected);
      selectedMesh.material.emissiveIntensity = 0.4;
      animatePlotElevation(selectedMesh, selectedMesh.userData.originalY + PLOT_HOVER_HE
      showPlotPanel(selectedMesh.userData.plotData);

      // Focus camera on selected plot
      focusCameraOnPlot(selectedMesh.position, camera, orbitControls);
    } else {
      hidePlotPanel();
    }
  }

  renderer.domElement.addEventListener('click',    handleSelect);
  renderer.domElement.addEventListener('touchend', handleSelect, { passive: true });
}

// Smooth elevation animation (no GSAP needed — requestAnimationFrame)
function animatePlotElevation(mesh, targetY, duration) {
  const startY  = mesh.position.y;
  const startMs = performance.now();
  function step(now) {
    const t = Math.min((now - startMs) / duration, 1);
    const ease = 1 - Math.pow(1 - t, 3);  // cubic ease-out
    mesh.position.y = startY + (targetY - startY) * ease;
    if (t < 1) requestAnimationFrame(step);
  }
  requestAnimationFrame(step);
}
```

OrbitControls Setup

```
// Load OrbitControls from CDN (add to head):
// <script src="https://cdn.jsdelivr.net/npm/three@0.128.0/examples/js/controls/OrbitCor

function setupControls(camera, renderer) {
  const controls = new THREE.OrbitControls(camera, renderer.domElement);
  controls.enableDamping    = true;
  controls.dampingFactor    = 0.08;
  controls.screenSpacePanning = false;
  controls.minDistance      = 6;
```

```
  controls.maxDistance      = 45;
  controls.maxPolarAngle    = Math.PI / 2.1;  // Don't go below ground
  controls.minPolarAngle    = 0.1;
  controls.rotateSpeed      = 0.7;
  controls.panSpeed         = 0.8;
  controls.zoomSpeed        = 1.0;
  controls.touches = {
    ONE:   THREE.TOUCH.ROTATE,
    TWO:   THREE.TOUCH.DOLLY_PAN,
  };
  return controls;
}

// Reset camera to default
function resetCamera(camera, controls) {
  animateCameraTo(camera, controls,
    { x: 0, y: 18, z: 20 },    // target position
    { x: 0, y: 0,  z: 0 },     // look-at target
    600
  );
}

// Focus camera on a specific plot
function focusCameraOnPlot(plotPos, camera, controls) {
  const offset = { x: plotPos.x + 3, y: plotPos.y + 10, z: plotPos.z + 8 };
  animateCameraTo(camera, controls, offset, plotPos, 700);
}
```

2D / SD Views

```
// 2D Mode: Orthographic camera looking straight down
function switchTo2D(scene, camera, renderer) {
  // Smoothly tilt camera to top-down view (polar angle → 0)
  controls.maxPolarAngle = 0.01;
  animateCameraTo(camera, controls, { x: 0, y: 30, z: 0 }, { x: 0, y: 0, z: 0 }, 800);

  // Lower plot heights to 0.05 (flat appearance)
  plotMeshes.forEach(m => {
    m.geometry = new THREE.BoxGeometry(PLOT_WIDTH, 0.05, PLOT_DEPTH);
    animatePlotElevation(m, 0.025, 500);
  });
  scene.fog = null;
}

// SD Mode: Satellite perspective (fixed angle, no orbit)
function switchToSD(scene, camera) {
  controls.maxPolarAngle = Math.PI / 4;
  controls.minPolarAngle = Math.PI / 4;
  animateCameraTo(camera, controls, { x: 0, y: 25, z: 15 }, { x: 0, y: 0, z: 0 }, 800);
}

// 3D Mode: Restore full 3D
function switchTo3D(scene, camera) {
  controls.maxPolarAngle = Math.PI / 2.1;
  controls.minPolarAngle = 0.1;
```

```
  scene.fog = new THREE.FogExp2(0x0f0f0f, 0.035);
  plotMeshes.forEach(m => {
    m.geometry = new THREE.BoxGeometry(PLOT_WIDTH, PLOT_HEIGHT, PLOT_DEPTH);
    animatePlotElevation(m, m.userData.originalY, 500);
  });
}
```

Animation Loop

```
function startAnimationLoop(renderer, scene, camera, controls) {
  let animFrameId;
  function animate() {
    animFrameId = requestAnimationFrame(animate);
    controls.update();  // Required for damping
    renderer.render(scene, camera);
  }
  animate();
  return () => cancelAnimationFrame(animFrameId);
}
```

CSS — COMPLETE RESPONSIVE STYLES

```
/* ═══════════════════════════════════════
   BASE RESET + FONTS
   ═══════════════════════════════════════ */
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
html {
  -webkit-text-size-adjust: none;
  text-size-adjust: none;
  -webkit-font-smoothing: antialiased;
}
body {
  font-family: 'Inter', system-ui, sans-serif;
  background: var(--color-bg);
  color: var(--color-text);
  overflow: hidden;          /* Canvas fills viewport */
  width: 100dvw;
  height: 100dvh;
  touch-action: none;        /* Let Three.js handle touch on canvas */
}

/* ═══════════════════════════════════════
   CANVAS
   ═══════════════════════════════════════ */
#canvas-container {
  position: fixed;
  inset: 0;
  z-index: 1;
}
#plot-canvas {
  display: block;
  width: 100%;
```

```
  height: 100%;
}

/* ═══════════════════════════════════════════
   TOP BAR
   ═══════════════════════════════════════════ */
.top-bar {
  position: fixed;
  top: 0; left: 0; right: 0;
  z-index: 100;
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: clamp(var(--space-3), 2vw, var(--space-5))
           clamp(var(--space-4), 3vw, var(--space-8));
  /* Gradient overlay — fades to transparent */
  background: linear-gradient(to bottom, rgba(15,15,15,0.92) 0%, transparent 100%);
  pointer-events: none;        /* Let clicks pass through gradient area */
}
.top-bar > * { pointer-events: all; } /* Re-enable on actual elements */

.brand { display: flex; align-items: center; gap: var(--space-3); }
.brand-logo {
  width: clamp(28px, 5vw, 40px);
  height: clamp(28px, 5vw, 40px);
  flex-shrink: 0;
}
.brand-text { display: flex; flex-direction: column; gap: 1px; }
.brand-name {
  font-family: 'Space Grotesk', sans-serif;
  font-size: var(--text-lg);
  font-weight: 700;
  letter-spacing: 0.02em;
  color: var(--color-text);
  line-height: 1.1;
}
.brand-sub {
  font-size: var(--text-xs);
  font-weight: 400;
  letter-spacing: 0.12em;
  color: var(--color-text-muted);
  text-transform: uppercase;
}

.btn-share {
  display: flex;
  align-items: center;
  gap: var(--space-2);
  padding: clamp(6px, 1.5vw, 10px) clamp(12px, 3vw, 20px);
  background: rgba(255,255,255,0.08);
  border: 1px solid rgba(255,255,255,0.12);
  border-radius: var(--radius-full);
  color: var(--color-text);
  font-size: var(--text-sm);
  font-weight: 500;
  backdrop-filter: blur(8px);
```

```
  -webkit-backdrop-filter: blur(8px);
  cursor: pointer;
  transition: background var(--transition), border-color var(--transition);
  white-space: nowrap;
}
.btn-share:hover  { background: rgba(255,255,255,0.14); }
.btn-share:active { background: rgba(255,255,255,0.20); }
.btn-share svg { width: 16px; height: 16px; }

/* ═══════════════════════════════════════
   PLOT INFO PANEL — DESKTOP (right side)
   ═══════════════════════════════════════ */
.plot-panel {
  position: fixed;
  top: clamp(60px, 8vh, 90px);
  right: clamp(var(--space-4), 2vw, var(--space-6));
  z-index: 200;
  width: clamp(240px, 28vw, 320px);
  max-width: calc(100vw - 2 * var(--space-4));
  background: rgba(22, 22, 22, 0.92);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-xl);
  backdrop-filter: blur(16px);
  -webkit-backdrop-filter: blur(16px);
  padding: clamp(var(--space-4), 3vw, var(--space-6));
  box-shadow: var(--shadow-panel);
  /* Slide-in animation */
  animation: slideInRight 0.3s cubic-bezier(0.16, 1, 0.3, 1) both;
}

.plot-panel[hidden] { display: none; }

@keyframes slideInRight {
  from { opacity: 0; transform: translateX(24px); }
  to   { opacity: 1; transform: translateX(0); }
}

.panel-close {
  position: absolute;
  top: var(--space-3);
  right: var(--space-3);
  width: 28px;
  height: 28px;
  min-width: 44px; min-height: 44px;
  display: grid; place-items: center;
  background: rgba(255,255,255,0.06);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-full);
  color: var(--color-text-muted);
  font-size: 1.25rem;
  cursor: pointer;
  transition: background var(--transition), color var(--transition);
}
.panel-close:hover { background: rgba(255,255,255,0.12); color: var(--color-text); }

.panel-plot-number {
```

```
  font-family: 'Space Grotesk', sans-serif;
  font-size: var(--text-xl);
  font-weight: 700;
  margin-bottom: var(--space-3);
}

/* Status badge */
.status-badge {
  display: inline-flex;
  align-items: center;
  gap: var(--space-2);
  padding: var(--space-1) var(--space-3);
  border-radius: var(--radius-full);
  font-size: var(--text-xs);
  font-weight: 600;
  letter-spacing: 0.06em;
  margin-bottom: var(--space-5);
}
.status-available { background: var(--badge-available-bg); color: var(--badge-available-
.status-sold      { background: var(--badge-sold-bg);      color: var(--badge-sold-text);
.status-dot {
  width: 7px; height: 7px;
  border-radius: 50%;
  background: currentColor;
  animation: pulse 2s ease-in-out infinite;
}
.status-sold .status-dot { animation: none; }
@keyframes pulse {
  0%, 100% { opacity: 1; } 50% { opacity: 0.4; }
}

/* Details grid */
.panel-details {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: var(--space-1);
  margin-bottom: var(--space-5);
}
.panel-detail-item {
  background: rgba(255,255,255,0.04);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-md);
  padding: var(--space-3);
}
.detail-label {
  display: block;
  font-size: var(--text-xs);
  color: var(--color-text-muted);
  letter-spacing: 0.08em;
  text-transform: uppercase;
  margin-bottom: var(--space-1);
}
.detail-value {
  font-size: var(--text-lg);
  font-weight: 600;
  font-variant-numeric: tabular-nums;
```

```
}
.detail-value.price { color: #ffffff; }
.detail-value small { font-size: var(--text-xs); color: var(--color-text-muted); font-wei

/* Action buttons */
.panel-actions {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: var(--space-3);
  margin-bottom: var(--space-4);
}
.btn-book, .btn-contact {
  padding: clamp(10px, 2vw, 13px) var(--space-4);
  border-radius: var(--radius-md);
  font-size: var(--text-sm);
  font-weight: 600;
  cursor: pointer;
  transition: all var(--transition);
  min-height: 44px;
  white-space: nowrap;
}
.btn-book {
  background: var(--color-accent);
  color: var(--color-cta-text);
  border: none;
  box-shadow: 0 2px 12px rgba(59, 130, 246, 0.4);
}
.btn-book:hover  { background: var(--color-accent-hover); transform: translateY(-1px); b
.btn-book:active { transform: translateY(0); }
.btn-contact {
  background: transparent;
  color: var(--color-text);
  border: 1px solid rgba(255,255,255,0.2);
}
.btn-contact:hover  { background: rgba(255,255,255,0.06); border-color: rgba(255,255,255
.btn-contact:active { background: rgba(255,255,255,0.10); }

/* Meta info */
.panel-meta { display: flex; flex-direction: column; gap: var(--space-2); }
.meta-item {
  display: flex;
  justify-content: space-between;
  align-items: center;
  font-size: var(--text-xs);
  padding-bottom: var(--space-2);
  border-bottom: 1px solid var(--color-divider);
}
.meta-item:last-child { border-bottom: none; padding-bottom: 0; }
.meta-item span:first-child { color: var(--color-text-muted); }
.meta-item span:last-child  { font-weight: 500; }

/* ═══════════════════════════════════════
   MOBILE — Panel becomes bottom sheet
   ═══════════════════════════════════════ */
@media (max-width: 640px) {
  .plot-panel {
```

```
    top: auto;
    right: 0; left: 0;
    bottom: clamp(110px, 18vh, 140px);  /* above bottom controls */
    width: 100%;
    max-width: 100%;
    border-radius: var(--radius-xl) var(--radius-xl) 0 0;
    animation: slideInUp 0.35s cubic-bezier(0.16, 1, 0.3, 1) both;
    max-height: 55vh;
    overflow-y: auto;
  }
  @keyframes slideInUp {
    from { opacity: 0; transform: translateY(40px); }
    to   { opacity: 1; transform: translateY(0); }
  }
  .panel-details { grid-template-columns: 1fr 1fr; }
}

/* ═══════════════════════════════════════════
   VIEW TOGGLE (2D / 3D / SD)
   ═══════════════════════════════════════════ */
.view-toggle {
  position: fixed;
  right: clamp(var(--space-4), 2vw, var(--space-6));
  bottom: clamp(140px, 22vh, 180px);
  z-index: 100;
  display: flex;
  flex-direction: column;
  gap: 2px;
  background: rgba(22,22,22,0.85);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-lg);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  overflow: hidden;
}
.view-btn {
  padding: clamp(8px, 1.5vw, 12px) clamp(10px, 2vw, 18px);
  min-width: 44px;
  min-height: 40px;
  font-size: var(--text-xs);
  font-weight: 600;
  color: var(--color-text-muted);
  background: transparent;
  border: none;
  cursor: pointer;
  letter-spacing: 0.04em;
  transition: background var(--transition), color var(--transition);
}
.view-btn:hover  { background: rgba(255,255,255,0.06); color: var(--color-text); }
.view-btn.active { background: rgba(255,255,255,0.12); color: var(--color-text); }

/* ═══════════════════════════════════════════
   RESET CAMERA BUTTON
   ═══════════════════════════════════════════ */
.btn-reset-cam {
  position: fixed;
```

```
  right: clamp(var(--space-4), 2vw, var(--space-6));
  bottom: clamp(95px, 14vh, 130px);
  z-index: 100;
  width: clamp(36px, 6vw, 48px);
  height: clamp(36px, 6vw, 48px);
  min-width: 44px;
  min-height: 44px;
  display: grid;
  place-items: center;
  background: rgba(22,22,22,0.85);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-full);
  color: var(--color-text-muted);
  cursor: pointer;
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  transition: background var(--transition), color var(--transition), transform var(--tra
}
.btn-reset-cam:hover  { background: rgba(255,255,255,0.10); color: var(--color-text); }
.btn-reset-cam:active { transform: rotate(180deg); }
.btn-reset-cam svg    { width: 18px; height: 18px; }

/* ═══════════════════════════════════════
   BOTTOM CONTROLS — FULL RESPONSIVE
   ═══════════════════════════════════════ */
.bottom-controls {
  position: fixed;
  bottom: 0; left: 0; right: 0;
  z-index: 100;
  padding: 0 clamp(var(--space-3), 3vw, var(--space-6));
  padding-bottom: clamp(var(--space-3), env(safe-area-inset-bottom, 12px), var(--space-6
  background: linear-gradient(to top, rgba(15,15,15,0.95) 0%, transparent 100%);
  display: flex;
  flex-direction: column;
  gap: var(--space-3);
  pointer-events: none;
}
.bottom-controls > * { pointer-events: all; }

/* Row 1: Status + Search */
.bottom-row-1 {
  display: flex;
  align-items: center;
  gap: var(--space-3);
}

/* Status Legend */
.status-legend { position: relative; flex-shrink: 0; }
.legend-toggle {
  display: flex;
  align-items: center;
  gap: var(--space-2);
  padding: clamp(8px, 1.5vw, 10px) clamp(10px, 2vw, 16px);
  min-height: 44px;
  background: rgba(22,22,22,0.85);
  border: 1px solid var(--color-border);
```

```
  border-radius: var(--radius-full);
  color: var(--color-text);
  font-size: var(--text-xs);
  font-weight: 600;
  letter-spacing: 0.08em;
  cursor: pointer;
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  transition: background var(--transition);
  white-space: nowrap;
}
.legend-toggle:hover { background: rgba(255,255,255,0.08); }
.legend-indicator {
  width: 10px; height: 10px;
  border-radius: 50%;
  background: var(--color-available);
  box-shadow: 0 0 6px var(--color-available);
  animation: pulse 2s ease-in-out infinite;
}

.legend-popup {
  position: absolute;
  bottom: calc(100% + 8px);
  left: 0;
  background: rgba(22,22,22,0.96);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-lg);
  padding: var(--space-3) var(--space-4);
  min-width: 160px;
  backdrop-filter: blur(16px);
  -webkit-backdrop-filter: blur(16px);
  box-shadow: var(--shadow-panel);
  animation: fadeInUp 0.2s ease both;
}
.legend-popup[hidden] { display: none; }

@keyframes fadeInUp {
  from { opacity: 0; transform: translateY(6px); }
  to   { opacity: 1; transform: translateY(0); }
}

.legend-item {
  display: flex;
  align-items: center;
  gap: var(--space-3);
  padding: var(--space-2) 0;
  font-size: var(--text-sm);
  color: var(--color-text-muted);
}
.legend-color {
  width: 12px; height: 12px;
  border-radius: 3px;
  flex-shrink: 0;
}

/* Search */
```

```
.search-wrapper {
  position: relative;
  flex: 1;
  max-width: 440px;
}
.search-icon {
  position: absolute;
  left: clamp(10px, 2vw, 14px);
  top: 50%;
  transform: translateY(-50%);
  width: 16px; height: 16px;
  color: var(--color-text-muted);
  pointer-events: none;
}
.search-input {
  width: 100%;
  height: clamp(40px, 7vw, 50px);
  padding-left: clamp(32px, 6vw, 42px);
  padding-right: var(--space-4);
  background: rgba(22,22,22,0.85);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-full);
  color: var(--color-text);
  font-size: var(--text-sm);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  outline: none;
  transition: border-color var(--transition), background var(--transition);
  /* Prevent iOS zoom on focus */
  font-size: max(16px, var(--text-sm));
}
.search-input::placeholder { color: var(--color-text-muted); }
.search-input:focus {
  border-color: rgba(255,255,255,0.3);
  background: rgba(30,30,30,0.92);
}

/* Search results dropdown */
.search-results {
  position: absolute;
  bottom: calc(100% + 6px);
  left: 0; right: 0;
  background: rgba(22,22,22,0.96);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-lg);
  overflow: hidden;
  max-height: 220px;
  overflow-y: auto;
  box-shadow: var(--shadow-panel);
}
.search-results[hidden] { display: none; }
.search-result-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: var(--space-3) var(--space-4);
```

```
  font-size: var(--text-sm);
  cursor: pointer;
  transition: background var(--transition);
  min-height: 44px;
}
.search-result-item:hover { background: rgba(255,255,255,0.06); }

/* Row 2: Bottom tabs */
.bottom-tabs {
  display: flex;
  gap: var(--space-2);
  justify-content: center;
  flex-wrap: nowrap;
}
.tab-btn {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: clamp(4px, 1vw, 8px);
  flex: 1;
  max-width: 200px;
  padding: clamp(10px, 2vw, 14px) clamp(var(--space-3), 3vw, var(--space-6));
  min-height: 44px;
  background: rgba(22,22,22,0.85);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-full);
  color: var(--color-text-muted);
  font-size: var(--text-sm);
  font-weight: 500;
  letter-spacing: 0.06em;
  cursor: pointer;
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  transition: all var(--transition);
  white-space: nowrap;
}
.tab-btn svg       { width: clamp(14px, 2.5vw, 18px); height: clamp(14px, 2.5vw, 18px); }
.tab-btn:hover     { background: rgba(255,255,255,0.08); color: var(--color-text); }
.tab-btn.active    { background: rgba(255,255,255,0.14); color: var(--color-text); borde
.tab-btn:active    { transform: scale(0.97); }

/* On very small screens: abbreviate tab labels */
@media (max-width: 380px) {
  .tab-btn span { display: none; }
  .tab-btn { max-width: 70px; padding: 12px; }
}

/* ═══════════════════════════════════════
   MODAL OVERLAY
   ═══════════════════════════════════════ */
.modal-overlay {
  position: fixed;
  inset: 0;
  z-index: 500;
  background: rgba(0,0,0,0.7);
  display: flex;
```

```
  align-items: flex-end;
  justify-content: center;
  padding: var(--space-4);
  backdrop-filter: blur(4px);
  -webkit-backdrop-filter: blur(4px);
  animation: fadeIn 0.2s ease both;
}
.modal-overlay[hidden] { display: none; }

@keyframes fadeIn {
  from { opacity: 0; } to { opacity: 1; }
}

.modal-card {
  width: 100%;
  max-width: 640px;
  max-height: 85dvh;
  background: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-xl) var(--radius-xl) var(--radius-lg) var(--radius-lg);
  overflow-y: auto;
  padding: var(--space-6);
  position: relative;
  animation: slideInUp 0.3s cubic-bezier(0.16, 1, 0.3, 1) both;
}
.modal-close {
  position: absolute;
  top: var(--space-4);
  right: var(--space-4);
  width: 44px; height: 44px;
  display: grid; place-items: center;
  background: rgba(255,255,255,0.06);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-full);
  color: var(--color-text-muted);
  font-size: 1.4rem;
  cursor: pointer;
  transition: background var(--transition);
}
.modal-close:hover { background: rgba(255,255,255,0.12); color: var(--color-text); }

/* ═══════════════════════════════════════
   TOAST NOTIFICATION
   ═══════════════════════════════════════ */
.toast {
  position: fixed;
  bottom: clamp(140px, 22vh, 180px);
  left: 50%;
  transform: translateX(-50%) translateY(20px);
  z-index: 600;
  padding: var(--space-3) var(--space-6);
  background: rgba(30,30,30,0.96);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-full);
  color: var(--color-text);
  font-size: var(--text-sm);
```

```
  font-weight: 500;
  pointer-events: none;
  opacity: 0;
  white-space: nowrap;
  transition: opacity 0.25s ease, transform 0.25s ease;
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
}
.toast.visible {
  opacity: 1;
  transform: translateX(-50%) translateY(0);
}

/* ═══════════════════════════════════════════
   RESPONSIVE ADJUSTMENTS
   ═══════════════════════════════════════════ */

/* Tablet (641px–1023px) */
@media (min-width: 641px) and (max-width: 1023px) {
  .plot-panel {
    width: clamp(260px, 35vw, 300px);
    top: clamp(70px, 10vh, 90px);
  }
  .view-toggle { bottom: clamp(160px, 24vh, 200px); }
  .btn-reset-cam { bottom: clamp(110px, 16vh, 150px); }
}

/* Large desktop (1440px+) */
@media (min-width: 1440px) {
  .plot-panel { width: 320px; }
  .bottom-row-1 { justify-content: center; }
  .search-wrapper { max-width: 500px; }
}

/* Landscape phone */
@media (max-height: 500px) and (orientation: landscape) {
  .plot-panel {
    top: 10px;
    max-height: calc(100dvh - 20px);
    overflow-y: auto;
  }
  .bottom-controls { padding-bottom: var(--space-2); }
  .tab-btn { padding: 8px 12px; min-height: 40px; }
  .brand-sub { display: none; }
}
```

JAVASCRIPT — COMPLETE FEATURE IMPLEMENTATION

Plot Panel Show/Hide

```
function showPlotPanel(plot) {
  const panel = document.getElementById('plot-panel');
  const isAvailable = plot.status === 'available' || plot.status === 'corner';

  document.getElementById('panel-number').textContent  = plot.number;
  document.getElementById('panel-area').textContent    = plot.area.toLocaleString();
  document.getElementById('panel-price').textContent   = plot.price;
  document.getElementById('panel-facing').textContent  = plot.facing;
  document.getElementById('panel-dims').textContent    = plot.dims;
  document.getElementById('panel-road').textContent    = plot.roadWidth + ' Road';

  const badge     = document.getElementById('panel-status-badge');
  const statusTxt = document.getElementById('panel-status-text');
  badge.className = `status-badge status-${isAvailable ? 'available' : 'sold'}`;
  statusTxt.textContent = isAvailable ? 'AVAILABLE' : 'SOLD';

  const btnBook = document.getElementById('btn-book-now');
  btnBook.disabled = !isAvailable;
  btnBook.style.opacity = isAvailable ? '1' : '0.4';

  panel.hidden = false;
}

function hidePlotPanel() {
  document.getElementById('plot-panel').hidden = true;
}
```

Search

```
function setupSearch(plotData, plotMeshes, camera, controls) {
  const input   = document.getElementById('plot-search');
  const results = document.getElementById('search-results');

  input.addEventListener('input', () => {
    const q = input.value.trim();
    if (!q) { results.hidden = true; return; }
    const matches = plotData.filter(p => p.number.toString().startsWith(q));
    if (!matches.length) { results.hidden = true; return; }
    results.innerHTML = matches.map(p => `
      <div class="search-result-item" data-id="${p.id}">
        <span>Plot #${p.number}</span>
        <span style="color:var(--color-${p.status === 'sold' ? 'sold' : 'available'}); fo
          ${p.status.toUpperCase()}
        </span>
      </div>
    `).join('');
    results.hidden = false;
  });

  results.addEventListener('click', (e) => {
    const item = e.target.closest('.search-result-item');
    if (!item) return;
```

```
    const plotId = Number(item.dataset.id);
    const mesh   = plotMeshes.find(m => m.userData.plotId === plotId);
    if (mesh) {
      selectPlot(mesh);
      focusCameraOnPlot(mesh.position, camera, controls);
    }
    input.value = '';
    results.hidden = true;
  });

  // Close on outside click
  document.addEventListener('click', (e) => {
    if (!e.target.closest('.search-wrapper')) results.hidden = true;
  });
}
```

View Toggle

```
function setupViewToggle() {
  document.querySelectorAll('.view-btn').forEach(btn => {
    btn.addEventListener('click', () => {
      document.querySelectorAll('.view-btn').forEach(b => {
        b.classList.remove('active');
        b.setAttribute('aria-pressed', 'false');
      });
      btn.classList.add('active');
      btn.setAttribute('aria-pressed', 'true');
      switch (btn.dataset.view) {
        case '2d': switchTo2D(scene, camera, renderer); break;
        case '3d': switchTo3D(scene, camera); break;
        case 'sd': switchToSD(scene, camera); break;
      }
    });
  });
}
```

Bottom Tabs (Gallery / Info / Locate)

```
const MODAL_CONTENT = {
  gallery: {
    title: 'Project Gallery',
    html: `
      <h2 style="font-family:'Space Grotesk',sans-serif; margin-bottom:1.5rem;">Project
      <p style="color:var(--color-text-muted); margin-bottom:1rem;">
        Preethi Estates — Premium plots in a prime location with excellent connectivity.
      </p>
      <div style="display:grid; grid-template-columns: repeat(auto-fill, minmax(200px,
        ${[1,2,3,4,5,6]
<span style="display:none">[^1_1][^1_10][^1_11][^1_12][^1_13][^1_14][^1_15][^1_16][^1_17][^1

<div align="center">⁂</div>

[^1_1]: image.jpg
```

```
[^1_2]: image-2.jpg
[^1_3]: https://spacer.land
[^1_4]: https://nsventures.in/show-project-virtual-tour-aerial-exterior-interior/
[^1_5]: https://iolitesoftwares.com/real-estate-booking-software.html
[^1_6]: https://www.pixready.com/blog/3d-virtual-tour-real-estate
[^1_7]: https://canibuild.com/en-us/resources/insights/3d-site-plan-software-guide/
[^1_8]: https://ecweb.ecer.com/topic/cn/detail-866548-threejs_powers_immersive_3d_maps_
[^1_9]: https://ones.software/in/bookings-one/interactive-map
[^1_10]: https://virtualspaces.tech/blog/property-management-ai-3d-floor-plans-reduce-va
[^1_11]: https://www.reddit.com/r/threejs/comments/1p4yjf5/interactive_3d_real_estate_we
[^1_12]: https://3d-mapper.com
[^1_13]: https://assets.nextleap.app/submissions/Urvashi_nextleap-03e5b865-64c9-4dc9-990e
[^1_14]: https://discourse.threejs.org/t/3d-apartment-with-different-styles-and-environm
[^1_15]: https://mapsplatform.google.com/maps-products/3d-maps/
[^1_16]: https://rend.pro/en/blog/3d-visualizations-of-plots/
[^1_17]: https://www.archilogic.com/project/building-a-3d-viewer-with-three-js-react-and
```